API Automation — third-party / open-source license inventory (all 88 tools)

What open-source or third-party engine sits behind each of the 88 tools in the API Automation Tools gallery, and under which license — so legal/compliance knows exactly what is embedded and what obligations apply.

> ***Method & honesty note.*** *This maps each tool to the libraries **actually bundled** in* `backend/package.json` / `frontend/package.json` *and used at runtime — not the tool's marketing name. Several tools are named after well-known OSS (Spectral, Promptfoo, 42Crunch, k6, Pact, Schemathesis); those are **concepts re-implemented in-house**, and are **not embedded**, so they carry **no license obligation** here. Verify versions at build time (e.g.* `license-checker`*) before shipping an attribution file.*

Executive summary — the posture is clean

- **No strong copyleft (GPL / AGPL / LGPL) is bundled** in the platform. In particular, **k6 (AGPL-3.0) is NOT embedded** — the "load" and "full-code export" tools *generate k6 (and Playwright) source for you to run in your own repo*; generating code in a language does not pull in that tool's license. Any k6/AGPL obligation falls on **your** use of the generated project, not on JBSIntelliQE.
- **Overwhelmingly permissive**: MIT, Apache-2.0, ISC, BSD — all allow commercial, closed-source redistribution with only attribution.
- **Two weak-copyleft items to track** (both fine for a SaaS, both file-level):
  - **axe-core — MPL-2.0** (via `@axe-core/playwright`, used by the cloud browser lab / UI checks).
  - **JSZip — dual MIT / GPL-3.0** (elect **MIT**; used for evidence/zip export).
- **Commercial dependency**: the AI behind most tools is the **Anthropic Claude API** (commercial terms, usage-metered). Optional **OpenAI / Azure OpenAI / Gemini** run on the customer's own keys and terms.
- **Action**: ship a `THIRD-PARTY-NOTICES` attribution file (Apache-2.0 §4 + MIT/BSD/ISC notices), keep axe-core's MPL notice, and elect MIT for JSZip in writing.

License legend (obligations in one line)

License	Type	Obligation
MIT / ISC / BSD-2/3	Permissive	Keep copyright + license text (attribution). No copyleft.
Apache-2.0	Permissive + patent grant	Attribution + state changes; includes an express patent license.
MPL-2.0	Weak copyleft (file-level)	Publish source of **modified MPL files only**; your own files stay closed.
Dual MIT/GPL	Elect one	Choose MIT → permissive. Record the election.
Commercial (Claude, etc.)	Proprietary SaaS	Pay per terms; no redistribution of the model.
Proprietary (JBSIntelliQE)	First-party	Our own code — no third-party obligation.

"Engine" key below: *In-house* = our own code (often Claude-powered logic), no third-party runtime engine. Where a real library does the work, it is named.

---

1) Generate & author (14 tools)

Almost entirely first-party logic driven by the Claude API; document-intake tools add permissive parsers.

Tool	Engine actually used	License
AI coworker (Atto)	In-house · **Anthropic Claude API**	Proprietary + Commercial
Autopilot (autonomous loop)	In-house · Claude API	Proprietary + Commercial
Story-driven generation	In-house · Claude API · `mammoth`, `pdf-parse` (doc intake)	Proprietary + Commercial; BSD-2 / MIT
Multimodal inputs	In-house · Claude API (vision)	Proprietary + Commercial
Author in plain English	In-house · Claude API	Proprietary + Commercial
Chat-to-test	In-house · Claude API	Proprietary + Commercial
Coverage-guided generation	In-house · Claude API	Proprietary + Commercial
Stateful / white-box generation	In-house · Claude API	Proprietary + Commercial
Model-based generation	In-house · Claude API	Proprietary + Commercial
Synthetic data factory	In-house · Claude API	Proprietary + Commercial
Flow builder	In-house	Proprietary
Step groups library	In-house	Proprietary
Unified cross-layer test	In-house · Playwright (UI) · `mssql`/`pg`/`mysql2` (DB)	Proprietary; Apache-2.0; MIT
Autonomous maintenance	In-house · Claude API	Proprietary + Commercial

2) Security & authorization (12 tools)

In-house security logic + Claude for judgment; schema checks via `ajv`; secrets via cloud SDKs.

Tool	Engine actually used	License
Security scan	In-house · Claude API · HTTP via `axios`/`undici`	Proprietary + Commercial; MIT
OWASP API Top-10	In-house (OWASP API 2023 ruleset)	Proprietary
Fuzz / property test	In-house	Proprietary
Sequence fuzzing	In-house	Proprietary
Authorization (BOLA/BFLA)	In-house (multi-role matrix)	Proprietary
Deep authorization & data exposure	In-house · Claude API	Proprietary + Commercial
Assertion strength (mutation)	In-house	Proprietary
Semantic assertion	In-house · Claude API (LLM judge)	Proprietary + Commercial
Data classification (PII/shadow)	In-house · Claude API	Proprietary + Commercial
OpenAPI security audit *(42Crunch-style)*	In-house · `ajv`, `yaml` — **42Crunch not embedded**	Proprietary; MIT / ISC
OAuth & secrets	`@aws-sdk/client-secrets-manager`, `@azure/keyvault-secrets`, `@google-cloud/secret-manager`	Apache-2.0 / MIT / Apache-2.0
Access control (RBAC/SSO/SCIM)	In-house · `jsonwebtoken` (OIDC)	Proprietary; MIT

3) Performance & resilience (6 tools)

Our own async HTTP load engine (Node + `undici`/`axios`). **k6 is a code-export target, not bundled.**

Tool	Engine actually used	License
Load test	In-house load engine · `undici`/`axios`	Proprietary; MIT
Load profile & SLA	In-house	Proprietary
Geo / distributed load	In-house (remote agents)	Proprietary
Distributed cloud load	In-house (remote agents)	Proprietary
Distributed load ramp	In-house (remote agents)	Proprietary

Tool	Engine actually used	License
Chaos / resilience	In-house (fault injection)	Proprietary

4) Contracts & virtualization (10 tools)

Schema/contract logic on `ajv` + `yaml`; servers on `express`. Pact/AsyncAPI are **our own implementations of the open specs**.

Tool	Engine actually used	License
Contract validation	`ajv`, `ajv-formats`, `yaml` · In-house	MIT / ISC
Contract drift	In-house · `ajv`	Proprietary; MIT
Regression baselines	In-house (snapshot + diff)	Proprietary
Breaking-change gate (spec diff)	In-house · `ajv`, `yaml`	Proprietary; MIT / ISC
Record & replay	In-house	Proprietary
Pact broker	In-house (Pact spec re-impl — `pact-js` **not** embedded)	Proprietary
AsyncAPI contracts	In-house (AsyncAPI spec) · `ajv`	Proprietary; MIT
Service virtualization	In-house · `express`	Proprietary; MIT
Hosted mock server	In-house · `express`	Proprietary; MIT
Contract proxy	In-house · `express`/`undici`	Proprietary; MIT

5) Coverage & intelligence (7 tools)

First-party analytics + Claude.

Tool	Engine actually used	License
Test coverage	In-house	Proprietary
Traffic coverage gaps	In-house	Proprietary
Test intelligence (flaky/quarantine)	In-house	Proprietary
Change impact	In-house	Proprietary
Suite optimizer	In-house	Proprietary
Semantic self-healing	In-house · Claude API	Proprietary + Commercial
Data-correctness verify	In-house · `mssql`/`pg`/`mysql2`	Proprietary; MIT

6) Protocols, data & AI (13 tools)

The most third-party-heavy group: real protocol clients for streaming, events, gRPC, DBs.

Tool	Engine actually used	License
Async & streaming (WS/SSE)	In-house (native WebSocket/SSE)	Proprietary
Events & gRPC	`kafkajs` · `@grpc/grpc-js`, `@grpc/proto-loader`, `grpc-reflection-js`	MIT / Apache-2.0
Callback verification	In-house · `express`	Proprietary; MIT
Data-driven testing	In-house · `xlsx` (table intake)	Proprietary; Apache-2.0
Test data profiles	In-house	Proprietary
Database validation	`mssql` · `mysql2` · `pg` (read-only SELECT)	MIT
Traffic capture	In-house (HAR/proxy capture)	Proprietary
Browser recorder	In-house (DevTools/HAR)	Proprietary
Trace correlation	In-house (Jaeger/Zipkin/Tempo REST) · `axios`	Proprietary; MIT

Tool	Engine actually used	License
LLM providers	In-house clients (OpenAI/Azure/Gemini, customer keys)	Proprietary + customer's provider terms
MCP server	In-house (Model Context Protocol, open spec)	Proprietary
AI / LLM & MCP testing	In-house · Claude API	Proprietary + Commercial
LLM eval harness *(Promptfoo-style)*	In-house · Claude API — **Promptfoo not embedded**	Proprietary + Commercial

7) Governance & compliance (7 tools)

Rule engines re-implemented in-house (Spectral-style), compliance posture logic, code export.

Tool	Engine actually used	License
API governance *(Spectral-style)*	In-house rules — **@stoplight/spectral not embedded**	Proprietary
Governance lint	In-house	Proprietary
Compliance packs (PCI/HIPAA/GDPR/PSD2)	In-house posture rules	Proprietary
AI-era compliance (EU AI Act/DORA)	In-house	Proprietary
AI cost / FinOps	In-house (token metering)	Proprietary
On-prem / air-gapped	In-house (deployment/policy)	Proprietary
Full-code & Monitoring-as-Code	In-house **code generators** — emit Playwright (Apache-2.0) & k6 (AGPL-3.0) *source as output*	Proprietary; generated code carries the target's license **for your repo**, not ours

8) Execution, CI & lifecycle (19 tools)

Delivery integrations via official GitHub/GitLab/Graph clients; execution via Playwright; reporting via Allure.

Tool	Engine actually used	License
Schedules & webhooks	In-house · `nodemailer` (alerts)	Proprietary; MIT
CI / GitHub Action	`@octokit/rest`	MIT
Release gate	In-house	Proprietary
Parallel run queue	In-house	Proprietary
Cloud browser lab	`@playwright/test` · `@axe-core/playwright` (**axe-core MPL-2.0**) · `pixelmatch`	Apache-2.0 / **MPL-2.0** / ISC
API + UI hybrid test	`@playwright/test` · DB drivers	Apache-2.0 / MIT
Monitoring	In-house	Proprietary
Public status page	In-house	Proprietary
Failure → fix workflow	In-house · Claude API	Proprietary + Commercial
Bug report	In-house · `@octokit/rest` / `@gitbeaker/rest` / MS Graph	Proprietary; MIT
Remediation PR	`@octokit/rest` · `@gitbeaker/rest`	MIT
Git-synced tests	`@octokit/rest` · `@gitbeaker/rest`	MIT
Evidence export	`pdf-lib` · `pdf-parse` · `archiver` · `jszip` (**dual MIT/GPL → MIT**)	MIT
Audit log	In-house · `mssql`	Proprietary; MIT
Traceability matrix	In-house	Proprietary
Test management	In-house	Proprietary
TM connectors (TestRail/Xray/Zephyr/qTest)	In-house REST clients · `axios`	Proprietary; MIT
Approvals	In-house	Proprietary
Collaboration	In-house	Proprietary

Appendix A — platform-wide libraries (not tool-specific) and their licenses

Backend runtime

Library	License	Library	License
express, express-rate-limit, helmet, cors	MIT	mssql, mysql2, pg	MIT
axios, undici, isomorphic-fetch	MIT	kafkajs	MIT
@playwright/test	Apache-2.0	@axe-core/playwright (axe-core)	**MPL-2.0**
@grpc/grpc-js, @grpc/proto-loader	Apache-2.0	grpc-reflection-js	Apache-2.0
ajv, ajv-formats	MIT	yaml, xpath, @xmldom/xmldom	ISC / MIT
allure-commandline, allure-playwright	Apache-2.0	xlsx (SheetJS)	Apache-2.0
mammoth	BSD-2-Clause	pdf-parse, pdf-lib, archiver	MIT
jszip	**MIT / GPL-3.0 (elect MIT)**	pixelmatch, pngjs	ISC / MIT
bcrypt, jsonwebtoken	MIT	zod, multer, nodemailer	MIT
@octokit/rest, @gitbeaker/rest	MIT	@microsoft/microsoft-graph-client	MIT
@aws-sdk/ , *@google-cloud/*	Apache-2.0	@azure/*	MIT

Frontend

Library	License	Library	License
react, react-dom, react-router-dom	MIT	recharts	MIT
axios, clsx, tailwind-merge	MIT	lucide-react	ISC
class-variance-authority	Apache-2.0	tailwindcss, vite, eslint	MIT

AI / model providers

Provider	Terms
Anthropic Claude API (primary)	Commercial, usage-metered (Anthropic Commercial Terms)
OpenAI / Azure OpenAI / Google Gemini (optional)	Customer's own keys + each provider's terms

---

Appendix B — compliance checklist before shipping

– Generate a machine `THIRD-PARTY-NOTICES.txt` at build (`license-checker --production` / `pnpm licenses`) and keep it in the release.

– Confirm **no GPL/AGPL/LGPL** crept in transitively (CI gate on `license-checker --failOn 'GPL;AGPL;LGPL'`).

– Record the **JSZip = MIT** election in writing.

– Keep the **axe-core MPL-2.0** notice; if you ever patch axe-core files, publish those files' source.

– State in docs that **generated k6 projects are AGPL-3.0** — the obligation is the customer's, on their repo.

– Keep Anthropic (and any OpenAI/Azure/Gemini) commercial terms current in the vendor register.